SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA06 – Design and Analysis of Algorithms

Assignment 3 – CO3: Divide and Conquer (Pair Work, 50 Marks)

Role	Name	Reg. No.
Student A	Deepak S	192511124
Student B	_____	_____

Course Outcome: CO3 – Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication.

How the experiments were done. Every algorithm below was implemented in Python 3 (source code in the Appendix) and run on a single-core machine. "Comparisons" and "multiplications" are counted by instrumenting the code, so they are exact and repeatable. Timings are the best of 2–3 runs in milliseconds; they depend on the machine and on Python's interpreter overhead, so only their trends should be read. Random data used a fixed seed (42). Every algorithm was checked against a trusted baseline (Python's built-in sort/min/max, NumPy matrix product, brute force) before any measurement was recorded.

Q1 – Decomposition and Elimination in Ordered Data

Part A – Finding Maximum and Minimum

Shared Group Work

- **Problem:** find both the smallest and the largest element of an array of n numbers. Finding them independently needs 2(n − 1) comparisons.
- **Strategy (tournament):** split the array into two halves, solve each half recursively and combine the two (min, max) answers.
- **Base cases:** one element → it is both min and max (0 comparisons); two elements → one comparison decides which is min and which is max.
- **What a subproblem returns:** the pair (min, max) of its segment.
- **Combine:** new min = smaller of the two minima (1 comparison); new max = larger of the two maxima (1 comparison).

```
MaxMin(a, lo, hi):
    if lo = hi:      return (a[lo], a[lo])
    if hi = lo + 1:  compare a[lo], a[hi]; return (smaller, larger)      // 1 comparison
    mid ← (lo + hi) / 2
    (min1, max1) ← MaxMin(a, lo, mid);   (min2, max2) ← MaxMin(a, mid+1, hi)
    return (min(min1, min2), max(max1, max2))                            // 2 comparisons
```

Implemented as maxmin() in the Appendix; the result was checked against Python's min() and max() for every dataset.

Student A (Deepak S) – Design and Mathematical Analysis

Decomposition for n = 8 (each box is a subproblem; the number in brackets is the comparisons spent when it is combined):

```
                    [0..7]                  combine: 2
                  /        \
            [0..3]          [4..7]          combine: 2 each
           /      \        /      \
```

```
     [0..1]    [2..3]      [4..5]    [6..7]   base case: 1 each

Total = 4 × 1 (pairs) + 2 × 2 + 1 × 2 = 10 = 3n/2 − 2
```

- **How subproblems are formed:** the index range [lo, hi] is cut at mid, giving two independent subproblems of size about n/2.
- **Combine operation:** compare the two minima and the two maxima: 2 comparisons, independent of the segment size.

Expected number of comparisons. With T(2) = 1 and T(n) = 2T(n/2) + 2:

```
T(n) = 2T(n/2) + 2 = 4T(n/4) + 4 + 2 = … = 2^(k−1)·T(2) + (2 + 4 + … + 2^(k−1))     (n = 2^k)
     = n/2 + (n − 2) = 3n/2 − 2
```

Comparison with independent search: finding max and min separately takes (n − 1) + (n − 1) = 2n − 2 comparisons. The difference is (2n − 2) − (3n/2 − 2) = n/2, so the divide-and-conquer method saves 25% of the comparisons for large n (ratio (3n/2 − 2)/(2n − 2) → 0.75). Example n = 1,024: 1,534 against 2,046. The saving arises because each pair is compared once to separate a "candidate minimum" from a "candidate maximum", so only half of the elements compete for each role.

Student B – Experimental Validation

Datasets: random values, already sorted, reverse sorted, and all-equal values (to test whether the count depends on the data), for sizes from 2 up to 100,000. Powers of two test the formula exactly; sizes 100, 1,000, 10,000 and 100,000 test the general case.

n	Predicted 3n/2 – 2	Observed D&C (all four datasets)	Independent min+max (observed)	D&C ÷ independent
2	1	1	2	0.500
4	4	4	6	0.667
8	10	10	14	0.714
16	22	22	30	0.733
64	94	94	126	0.746
256	382	382	510	0.749
1,024	1,534	1,534	2,046	0.750
4,096	6,142	6,142	8,190	0.750
65,536	98,302	98,302	131,070	0.750
100	148	162	198	0.818
1,000	1,498	1,510	1,998	0.756
10,000	14,998	15,902	19,998	0.795
100,000	149,998	165,534	199,998	0.828

Observations. (1) For n = 2^k the observed count equals 3n/2 − 2 exactly, on all four datasets, so the count does not depend on the data order. (2) The D&C count always stays below 2n − 2, and the ratio tends to 0.75.

Discrepancy. For n that is not a power of two the observed value is higher than 3n/2 − 2 (n = 100: 162 against 148; n = 100,000: 165,534 against 149,998, about 10% more). Reason: the formula assumes every leaf of the recursion is a pair. When n is split at the middle repeatedly, some leaves are single elements; a single element costs no comparisons but its parent still pays the full 2 comparisons of the combine step. The count is therefore not 3n/2 − 2 but still stays below 2n − 2. This can be removed by pairing the elements first

(n/2 comparisons) and then taking the min of the smaller ones and the max of the larger ones, which gives exactly 3n/2 − 2 for every n.

Part B – Binary Search

Shared Group Work

Binary search finds a target in a **sorted** array by comparing it with the middle element and discarding the half that cannot contain it.

```
BinarySearch(a, key):
    lo ← 0;  hi ← n − 1
    while lo ≤ hi:
        mid ← (lo + hi) / 2
        if a[mid] = key:  return mid
        if a[mid] < key:  lo ← mid + 1          // discard left half and mid
        else:             hi ← mid − 1          // discard right half and mid
    return NOT FOUND
```

Student A – Experimental Investigation

Test cases: successful searches for **every** element (n ≤ 100,000; a random sample of 20,000 keys for n = 10^6) and unsuccessful searches using keys that lie between elements (the array holds the even numbers 0, 2, 4, …, and odd keys are searched). One "comparison" means one probe of a[mid].

n	Successful: average	Successful: maximum	Unsuccessful: average	Unsuccessful: maximum	Bound $\log_2 n + 1$
7	2.43	3	3.00	3	3
15	3.27	4	4.00	4	4
100	5.80	7	6.74	7	7
1,000	8.99	10	9.98	10	10
10,000	12.36	14	13.36	14	14
100,000	15.69	17	16.69	17	17
1,000,000	18.95	20	19.95	20	20

Evidence. The maximum never exceeded $\log_2 n + 1$, and it is reached by unsuccessful searches (n = 10^6: 20 probes, against a possible 1,000,000 for a linear scan). Every time n grows tenfold, the probe count grows by only about 3.3 ($\log_2 10$), e.g. average 8.99 → 12.36 → 15.69 → 18.95. Successful searches need about one probe fewer on average than unsuccessful ones, since a successful search can stop early at a middle element. For n = 7 and 15 (2^k − 1) the maximum is exactly 3 and 4.

Student B – Elimination Logic

- **Decision rule:** compare key with a[mid]. Equal → found. key > a[mid] → continue in the right part (lo = mid + 1). key < a[mid] → continue in the left part (hi = mid − 1).
- **What is eliminated:** after each comparison, the middle element and one whole half of the current range, i.e. about half of the remaining search space.
- **Why the target cannot be in the eliminated part:** the array is sorted ascending. If key > a[mid], every element at indices ≤ mid is ≤ a[mid] < key, so none of them can equal key. The argument for key < a[mid] is symmetric. This is the only place where sortedness is needed, and it is also why binary search is wrong on unsorted data.

- **Termination:** each iteration either returns, or shrinks the range hi − lo + 1 to at most half its size; the loop stops when the range is empty (lo > hi), which means every position has been eliminated and the key is absent.
- **Efficiency:** the range halves at every step, n → n/2 → n/4 → … → 1, so at most $\log_2 n + 1$ steps are needed: T(n) = T(n/2) + 1 = Θ(log n). The work spent per elimination step is constant, so doubling the data adds only one more comparison.

Individual verification note: *Student A can explain the elimination argument above, and Student B can explain the tournament decomposition in Part A.*

Q2 – The Quality of Decomposition in Sorting

Shared Group Work

- **Merge Sort:** split at the middle, sort both halves, merge (stable, ties taken from the left half).
- **Quick Sort pivot strategy (clearly stated):** the **first element** of the sub-array is the pivot. The partition is stable and out-of-place (elements smaller than the pivot, then elements greater than the pivot, in their original order). Each partition of m elements costs m − 1 comparisons. All values are distinct. A second variant, **median-of-three** (median of first, middle and last elements, 3 extra comparisons per partition), is measured for comparison.
- **Experiment:** n = 1,023 elements (2^10 − 1, so that a perfectly balanced recursion tree exists) on four inputs; then growing n for the sorted and random inputs. We record partition sizes, comparisons and the maximum recursion depth (the number of levels in the recursion tree).

Student A (Deepak S) – Quick Sort Investigation

Input giving balanced partitions. With a first-element pivot, the pivot must be the median of its sub-array. The input is built recursively: place the median first, then a balanced arrangement of the smaller half, then of the larger half. Every partition then splits exactly into (m − 1)/2 and (m − 1)/2 elements.

Input giving severely unbalanced partitions. An already sorted array (ascending or descending): the first element is the minimum (or maximum), so the split is always 0 and m − 1. A "nearly sorted" array behaves almost the same.

Input (n = 1,023)	First partitions: size → (left, right)	Comparisons	Max depth	Median-of-3: comparisons / depth
Balanced (constructed)	1023 → (511, 511) 511 → (255, 255) 511 → (255, 255)	8,194	10	9,727 / 10
Random	1023 → (154, 868) 154 → (124, 29) 868 → (209, 658)	10,734	21	11,503 / 18
Sorted ascending	1023 → (0, 1022) 1022 → (0, 1021) 1021 → (0, 1020)	522,753	1023	9,727 / 10
Sorted descending	1023 → (1022, 0) 1022 → (1021, 0) 1021 → (1020, 0)	522,753	1023	9,727 / 10

Growth with n (first-element pivot):

n	Sorted input: comparisons	n(n−1)/2	Sorted: max depth	Random input: comparisons	Random: max depth	$n \cdot \log_2 n$

250	31,125	31,125	250	1,979	15	1,991
500	124,750	124,750	500	4,408	16	4,483
1,000	499,500	499,500	1,000	10,957	22	9,966
2,000	1,999,000	1,999,000	2,000	25,751	23	21,932
4,000	7,998,000	7,998,000	4,000	56,781	28	47,863

Findings. On the sorted input the comparisons equal n(n − 1)/2 exactly (n = 4,000: 7,998,000), the recursion depth equals n, and every partition is (0, m − 1). On the balanced input the depth is 10 = $\log_2(1{,}024)$ and each partition splits equally; random input lies in between (depth 21 for n = 1,023, 28 for n = 4,000, i.e. about 2–3 $\log_2 n$). For n = 4,000 the sorted case costs about 140 times as many comparisons as the random one (7,998,000 against 56,781).

How pivot selection influences decomposition. The pivot decides where the array is cut. A pivot near the median gives two subproblems of about m/2; an extreme pivot gives one empty subproblem and one of size m − 1, which is no real division. A fixed rule (the first element) is predictable, so a particular input (sorted data, common in practice) can always defeat it. Median-of-three cuts sorted input at the middle (9,727 comparisons and depth 10 for n = 1,023, against 522,753 and 1,023) because the middle element of sorted data is the true median.

Student B – Comparative D&C Analysis

- **Merge Sort decomposition:** it cuts by position, not by value: the sub-array of size m is always cut into $\lfloor m/2 \rfloor$ and $\lceil m/2 \rceil$. The recursion tree therefore has the same shape for every input: depth $\lfloor \log_2 n \rfloor + 1$ = 11 levels for n = 1,023 (13 for n = 4,000), which the experiment confirmed on all four inputs. The data only influences the merge work: 5,110 to 8,925 comparisons for n = 1,023, roughly between $(n/2)\log_2 n$ and $n \cdot \log_2 n$.
- **Comparison with Quick Sort:** Quick Sort does its work in the **divide** step (partition) and nothing in the combine step; Merge Sort does nothing in the divide step and all the work in the **combine** step (merge). Quick Sort cuts by value, so the shape of its tree depends on the data (depth 10 to 1,023 in our runs); Merge Sort's tree does not. Quick Sort is in-place (O(log n) stack when the smaller part is recursed first), while Merge Sort needs O(n) extra memory and is stable.
- **Why Merge Sort is more predictable:** its partition quality is guaranteed by construction (equal halves), so $T(n) = 2T(n/2) + n = \Theta(n \log n)$ in every case. Quick Sort's partition quality is determined by the data.
- **Partition quality and tree height:** the work in each level of the tree is at most n comparisons, so total work ≈ n × height. A balanced tree has height $\log_2 n$; if each partition keeps a fraction α of the elements in the larger part, the height is log_{1/α} n. For a 90 : 10 split (α = 0.9), height = ln n / ln(10/9) ≈ 9.5 ln n, i.e. 6.6 $\log_2 n$ levels for the worst branch; the total is about 2.1 n $\log_2 n$, still O(n log n) but with a larger constant. For the extreme 0 : (m − 1) split the height is n.
- **Repeated imbalance gives $O(n^2)$:** if every partition removes only a constant number of elements, $T(n) = T(n-1) + (n-1) = n(n-1)/2 = \Theta(n^2)$, because there are n levels and each still costs about n comparisons. Our measurements confirm it: 522,753 = 1,023 × 1,022 / 2.

Joint Design Question

"If you were designing a sorting component for an application in which the characteristics of the input data are not known in advance, what information would you want before deciding between these two D&C strategies?"

Information needed	Why it matters	Leans towards

Is the data already (nearly) sorted or in a regular pattern? Can it be adversarial?	Fixed-pivot Quick Sort degrades to $\Theta(n^2)$ on sorted input (522,753 vs 8,194 comparisons).	Merge Sort, or Quick Sort with random / median-of-three pivot
Is a worst-case time guarantee needed (real-time, deadline)?	Merge Sort is $\Theta(n \log n)$ always; Quick Sort only in expectation.	Merge Sort
Must equal keys keep their original order (stability)?	Our merge is stable; partitioning in place is not.	Merge Sort
How much memory is available? Does the data fit in RAM?	Merge Sort needs O(n) extra; Quick Sort O(log n). Merge-based methods also work for external (disk) sorting.	Quick Sort if memory is tight; Merge Sort for external data
Data type and size: arrays or linked lists, record size, n?	Quick Sort is cache-friendly on arrays; merging works well on linked lists (no extra space) and is easy to parallelise.	Quick Sort (arrays); Merge Sort (lists)
Are there many duplicates?	A two-way partition on many equal keys gives unbalanced splits unless a three-way partition is used.	Merge Sort or 3-way Quick Sort

Conclusion: without information about the input I would choose Merge Sort for guaranteed behaviour (or randomised/median-of-three Quick Sort, optionally switching to Heap Sort when the depth grows too large, if memory is the main constraint).

Q3 – Breaking the Quadratic Barrier: Karatsuba Multiplication

Shared Group Work

- **Number representation:** non-negative integers stored as lists of decimal digits (least significant first). A "multiplication" in the counts below is one single-digit × single-digit product.
- **Standard (school) multiplication:** multiply every digit of x by every digit of y: n^2 single-digit multiplications for two n-digit numbers.
- **Karatsuba:** recursive, with the three-product trick below. Variants measured: recursing down to single digits (threshold 1) and hybrids that use school multiplication below 16 or 64 digits.
- **Crossover experiment:** random n-digit numbers with n = 4 … 2,048, product checked against Python's own big-integer multiplication, counts and repeated timings recorded.

Student A (Deepak S) – Algorithmic Derivation

- **Decomposition:** an n-digit number is split at m = n/2 digits: $x = x_1 \cdot 10^{m} + x_0$ and $y = y_1 \cdot 10^{m} + y_0$, where x_0, y_0 are the low halves and x_1, y_1 the high halves. Then $x \cdot y = x_1y_1 \cdot 10^{2m} + (x_1y_0 + x_0y_1) \cdot 10^{m} + x_0y_0$.
- **Subproblems:** the direct formula needs four half-size products (x_1y_1, x_1y_0, x_0y_1, x_0y_0): $T(n) = 4T(n/2) + O(n) = \Theta(n^2)$, no gain over school multiplication.
- **Why Karatsuba needs only three:** let $z_2 = x_1y_1$, $z_0 = x_0y_0$ and $z_1 = (x_1 + x_0)(y_1 + y_0) - z_2 - z_0$. Expanding, $(x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$, so $z_1 = x_1y_0 + x_0y_1$, the middle term, obtained from one multiplication plus additions and subtractions. Result: $x \cdot y = z_2 \cdot 10^{2m} + z_1 \cdot 10^{m} + z_0$. The three products are z_0, z_2 and $(x_1 + x_0)(y_1 + y_0)$. The extra work consists only of additions, subtractions and shifts, which are O(n).
- **Recurrence:** $T(n) = 3T(n/2) + cn$, $T(1) = 1$.
- **Solution:** Master Theorem with a = 3, b = 2, $f(n) = cn$: $n^{\log_2 3} = n^{1.585}$ grows faster than n, so Case 1 applies: $T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. By expansion: the recursion tree has levels k = 0 … $\log_2 n$, level k has 3^k subproblems of size $n/2^k$, costing $cn \cdot (3/2)^k$; the sum is dominated by the last level, $3^{\log_2 n} = n^{\log_2 3}$.

- **Interpretation:** doubling n multiplies the work by 3 (instead of 4). For n = 1,024: 3^10 = 59,049 single-digit products against 1,048,576, a reduction of 17.8 times; the measured counts below match these values exactly.

Student B – Crossover Investigation

Input sizes: powers of two from 4 to 2,048 digits (so that 3^k is exact). **Repeats:** 3 runs for n ≤ 512 and 2 runs for larger n; the minimum time is reported.

n (digits)	School: multiplications	Karatsuba: multiplications	Ratio school ÷ Karatsuba	Time ms: school	Time ms: Karatsuba (to 1 digit)	Time ms: hybrid (<16)	Time ms: hybrid (<64)
4	16	9	1.8	0.004	0.033	0.004	0.004
8	64	27	2.4	0.006	0.096	0.007	0.006
16	256	81	3.2	0.017	0.28	0.018	0.018
32	1,024	243	4.2	0.057	0.94	0.065	0.058
64	4,096	729	5.6	0.22	2.71	0.22	0.21
128	16,384	2,187	7.5	0.81	8.16	0.73	0.71
256	65,536	6,561	10.0	3.52	25.0	2.38	2.24
512	262,144	19,683	13.3	15.9	75.3	7.44	7.17
1,024	1,048,576	59,049	17.8	67.4	226.5	23.4	22.5
2,048	4,194,304	177,147	23.7	269.7	696.2	72.6	71.5

- **Multiplication counts** are exactly n^2 and $3^{(\log_2 n)}$: the ratio grows from 1.8 (n = 4) to 23.7 (n = 2,048).
- **Crossover range (time):** pure Karatsuba recursing to single digits is slower than school multiplication throughout the measured range (n = 2,048: 696 ms against 270 ms). The time ratio improves with each doubling (12.5×, 10.0×, 7.1×, 4.7×, 3.4×, 2.6× slower at n = 64 … 2,048). Extrapolating this trend, the pure version would break even only at roughly 25,000 digits (an extrapolation, not a measurement). A **hybrid** that switches to school multiplication below 64 digits is the same speed at n = 64 and faster from **about n = 128** (0.71 against 0.81 ms), 1.6× at 256 digits, 2.2× at 512, 3.0× at 1,024 and 3.8× at 2,048. The hybrid with threshold 16 breaks even at about 64–128 digits. Approximate crossover for this implementation: **64–128 digits**.
- **Why Karatsuba can be worse for small inputs:** each Karatsuba step needs additions of two half-size numbers, two subtractions, list slicing and shifting, all costing O(n) with a large constant, while school multiplication at small n is a tight double loop. When n is small, the saved sub-multiplications are cheaper than this overhead.
- **Recursion and decomposition overhead:** for n digits, pure Karatsuba makes $3^{(\log_2 n)}$ recursive calls (177,147 leaf calls at n = 2,048 and ≈ 266,000 calls overall), each allocating new lists. Switching to school multiplication for small blocks removes the lowest levels, which contain most of the calls.

Joint Conclusion

"Why does better asymptotic complexity not automatically imply better performance for every input size?" Big-O describes how the cost grows when n becomes large and hides constant factors and lower-order terms. Karatsuba's real cost is cn^1.585 with a large c (additions, subtractions, memory allocation, recursion calls), school multiplication's is about $c'n^2$ with a small c'. The asymptotically better method wins only when n^0.415 > c/c', which happens at some finite crossover size. Our measurements show this: the count of multiplications favours Karatsuba from n = 4, but time favours it only above about 64–128 digits (hybrid) and far above that for the pure version. Practical libraries therefore use hybrids with tuned thresholds.

Q4 – The Reality of Strassen's Matrix Multiplication

Shared Group Work

- **Block multiplication:** for n × n matrices (n even) split every matrix into four n/2 × n/2 blocks. The conventional method then uses eight block products.
- **Implementations (Appendix):** conventional triple loop (n^3 multiplications), conventional block recursion (8 sub-products, checked against NumPy; see the additional check in the Appendix), and Strassen's recursion (7 sub-products) with an adjustable cut-off below which the conventional loop is used, plus padding to the next power of two.
- **Correctness verification:** Strassen's result was compared with NumPy's product on random integer matrices of sizes 1, 2, 3, 5, 7, 10, 33 and 100 (non-powers of two use padding), both for full recursion and cut-off 4; all results were identical. The sample A = [[1, 7], [3, 5]], B = [[1, 3], [7, 5]] gives C = [[50, 38], [38, 34]].

Student B – Algebraic Design and Correctness

Conventional block decomposition. With A = [A11 A12; A21 A22] and B = [B11 B12; B21 B22]:

```
C11 = A11·B11 + A12·B21        C12 = A11·B12 + A12·B22
C21 = A21·B11 + A22·B21        C22 = A21·B12 + A22·B22
```

These are **eight** block multiplications (A11B11, A12B21, A11B12, A12B22, A21B11, A22B21, A21B12, A22B22), giving $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.

Transformation to seven. Strassen forms seven products from sums and differences of blocks:

```
M1 = (A11 + A22)(B11 + B22)    M2 = (A21 + A22) B11       M3 = A11 (B12 − B22)
M4 = A22 (B21 − B11)             M5 = (A11 + A12) B22       M6 = (A21 − A11)(B11 + B12)
M7 = (A12 − A22)(B21 + B22)

C11 = M1 + M4 − M5 + M7     C12 = M3 + M5     C21 = M2 + M4     C22 = M1 − M2 + M3 + M6
```

The extra work is 10 block additions/subtractions before the products and 8 after (18 in all), each $\Theta(n^2)$. Since matrices are multiplied through the ring operations only (no commutativity is used), the formulas hold for blocks as well as for numbers.

Why the reconstructed expressions are correct (every term expanded; blocks keep their order):

- C12 = M3 + M5 = A11B12 − A11B22 + A11B22 + A12B22 = A11B12 + A12B22
- C21 = M2 + M4 = A21B11 + A22B11 + A22B21 − A22B11 = A21B11 + A22B21
- C11 = M1 + M4 − M5 + M7 = (A11B11 + A11B22 + A22B11 + A22B22) + (A22B21 − A22B11) − (A11B22 + A12B22) + (A12B21 + A12B22 − A22B21 − A22B22) = A11B11 + A12B21 (all other terms cancel)
- C22 = M1 − M2 + M3 + M6 = (A11B11 + A11B22 + A22B11 + A22B22) − (A21B11 + A22B11) + (A11B12 − A11B22) + (A21B11 + A21B12 − A11B11 − A11B12) = A22B22 + A21B12

Numerical check on the sample (A11 = 1, A12 = 7, A21 = 3, A22 = 5; B11 = 1, B12 = 3, B21 = 7, B22 = 5): M1 = 36, M2 = 8, M3 = −2, M4 = 30, M5 = 40, M6 = 8, M7 = 24, giving C11 = 50, C12 = 38, C21 = 38, C22 = 34, identical to the conventional product.

Padding. The recursion needs an even size at every level. If n is not a power of two, both matrices are padded with zero rows and columns to the next power of two m ≥ n; the product of the padded matrices has the required product in its top-left n × n block, because the extra zeros contribute nothing to any sum; the extra rows and columns are then cut off. (Alternatively, pad by one row/column only at the levels where the size is odd.) The padding costs extra work: n = 65 is padded to 128, and Strassen then needs 7^7 = 823,543 multiplications compared with 65^3 = 274,625 for the conventional method, three times more.

Student A (Deepak S) – Experimental and Engineering Analysis

Matrix sizes: n = 4 … 128 for the pure-Python implementation (so that all overheads are visible) and n = 256 … 2,048 for a NumPy implementation whose base case is the optimised BLAS product. Random matrices; 3 repeats (1 for n = 128); the minimum time is reported.

Multiplication counts (exact):

n	Conventional n^3	Strassen to 1×1: $7^{\log_2 n}$	Ratio	Hybrid (cut-off 8)	Hybrid (cut-off 16)
4	64	49	0.766	64	64
8	512	343	0.670	512	512
16	4,096	2,401	0.586	3,584	4,096
32	32,768	16,807	0.513	25,088	28,672
64	262,144	117,649	0.449	175,616	200,704
128	2,097,152	823,543	0.393	1,229,312	1,404,928

Measured time, pure Python (ms):

n	Conventional	Strassen (to 1×1)	Strassen (cut-off 8)	Strassen (cut-off 16)
4	0.007	0.11	0.007	0.007
8	0.031	0.81	0.029	0.029
16	0.17	5.76	0.30	0.16
32	0.99	41.1	2.46	1.48
64	6.86	307.3	19.6	12.2
128	58.9	2,125	151.4	95.9

Measured time with NumPy blocks and BLAS base case (ms):

n	BLAS product (A @ B)	Strassen, cut-off 32	cut-off 64	cut-off 128	cut-off 256	Max. error vs BLAS
256	0.53	2.90	1.25	0.78	–	2.1e-13
512	3.28	21.4	9.82	7.25	5.33	8.5e-13
1024	24.3	161.8	84.5	62.7	45.2	3.6e-12
2048	183.7	1,155	589.0	443.4	347.3	1.6e-11

- **Counts:** Strassen reduces multiplications by 2.5× at n = 128 for full recursion (823,543 against 2,097,152; 3.8× at n = 1,024); the hybrid with cut-off 16 keeps a 1.5× reduction (1,404,928).
- **Practical crossover: none was observed in the tested range.** In pure Python the conventional method is faster at every size up to n = 128 (58.9 ms against 95.9 ms for the best hybrid; 2,125 ms for full recursion, 36 times slower). With NumPy and an optimised BLAS base case the conventional product is faster up to n = 2,048 (183.7 ms against 347.3 ms for the best cut-off of 256). The best cut-off is the largest one tested, which shows that the benefit of reducing multiplications is smaller than the cost of the extra operations. The crossover for this implementation therefore lies beyond about n = 2,000 (outside our test range); I do not extrapolate a number. Highly tuned libraries report crossovers in the range of a few hundred to a few thousand depending on the hardware.
- **Addition/subtraction overhead:** 18 matrix additions per level, i.e. $18(n/2)^2$ operations. Counting additions as well as multiplications, full Strassen at n = 1,024 does about 1.97×10^9 operations against 2.15×10^9 for the conventional method (only about 8% fewer), so the multiplication saving mostly disappears (calculation: additions $6(7^{10} - 4^{10})$ = 1,688,560,038 plus multiplications 7^{10} = 282,475,249).

- **Recursion overhead:** full recursion makes 7^k calls, each building new lists (slicing, joining). The hybrid with cut-off 16 is 22 times faster than full recursion at n = 128 (95.9 ms against 2,125 ms), showing how much of the time is spent in the lowest levels.
- **Padding overhead:** as computed above, an n slightly above a power of two can cost 3–4 times as much (n = 65 → 128). Padding also increases memory by up to four times.
- **Memory/cache effects:** Strassen allocates 10 sums and 7 products per level as temporary matrices, which adds memory of the order of the input size, and its additions are memory-bound streaming operations. BLAS keeps blocks in cache and uses vector instructions, which explains why the NumPy base case is hard to beat; the gap between BLAS and Strassen (1.9×) did not shrink between n = 1,024 and n = 2,048. The maximum difference from the BLAS result (3.6×10^{-12} at n = 1,024, 1.6×10^{-11} at n = 2,048) is small but grows with n: Strassen is numerically a little less accurate in floating point.

Joint Engineering Decision

"Under what conditions would you use conventional matrix multiplication even though Strassen has superior asymptotic complexity?"

- When n is small or moderate (below the measured crossover, which we did not reach below n = 2,048 in this experiment), or when the matrices are the small blocks of a larger computation.
- When an optimised library (BLAS/GPU) is available, since it uses cache blocking and SIMD, and is hard to beat.
- When n is not a power of two (and especially just above one), because padding wastes work and memory.
- When memory is limited, because Strassen needs temporary matrices.
- When numerical accuracy matters (floating-point error is slightly larger), or when the matrices are sparse or structured (specialised algorithms are better).
- Strassen (with a cut-off to the conventional method) is worth using only for very large dense matrices on hardware where the saved multiplications outweigh the extra additions.

Q5 – Upgrading Closest Pair Using Divide and Conquer

Shared Group Work

- **Brute force (from CO2):** compute the distance of every pair: n(n − 1)/2 distance calculations, $\Theta(n^2)$.
- **D&C design:** sort the points by x once; recursively find the closest pair in the left and right halves (δ = the smaller of the two distances); then check the pairs that straddle the dividing line within the strip of width 2δ. Sub-arrays of up to 3 points are solved by brute force.

```
ClosestPair(P sorted by x):  returns (best distance δ, the points sorted by y)
    if |P| ≤ 3:  brute force; sort by y; return
    mid ← |P|/2;  midx ← x of P[mid]
    (δL, YL) ← ClosestPair(left half);   (δR, YR) ← ClosestPair(right half)
    δ ← min(δL, δR);   Y ← merge(YL, YR) by y                       // O(n)
    strip ← points of Y with |x − midx| < δ
    for each point p in strip, in y order:
        compare p only with the following points q with q.y − p.y < δ   // at most 7 (in
practice ~5)
    return (δ, Y)
```

Both programs were checked against each other on every dataset; their distances were identical in every case.

Student A (Deepak S) – Decomposition Design

- **How the points are organised:** sorted by x-coordinate once at the start (Px). Each recursive call also returns its points sorted by y-coordinate, built by merging the two halves' y-lists (as in Merge Sort), so that no re-sorting is needed.
- **How the problem is divided:** the x-sorted array is cut at the median index; the vertical line through the middle point separates the left and right halves, each containing about n/2 points.
- **What each recursive call returns:** the smallest distance within its set of points (and the pair achieving it) and the set's points sorted by y.
- **Combine step:** $\delta = \min(\delta L, \delta R)$. The closest pair overall is either entirely in one half (distance $\geq \delta$ already known) or **straddles** the line. A straddling pair closer than δ must have both points within δ of the line, so only the strip of width 2δ is examined, in y order.
- **Why the boundary needs special treatment:** the recursive calls never compare a left point with a right point, so a close pair separated by the line is invisible to both of them. Several points may also lie on or very near the dividing line itself, so the strip test uses the distance to the line, not left/right membership. The test with all points on one vertical line (n = 2,000) confirms this: the strip then contains nearly all the points, yet the algorithm needed only 1,272 distance calculations against 1,999,000.

Recurrence: $T(n) = 2T(n/2) + O(n)$ (the merge and the strip scan are linear) = $\Theta(n \log n)$, against $\Theta(n^2)$ for brute force.

Student B – Comparative Evidence

Equivalent datasets: the same randomly generated points (uniform on a 10,000 × 10,000 square) are given to both algorithms for n = 10 … 4,000, plus a degenerate dataset (all points on the line x = 5). **Recorded:** distance calculations (exact) and time in ms (best of 3 runs for n ≤ 1,000; a single run for larger n).

n	Brute force: distance calculations	D&C: distance calculations	Brute ÷ D&C	Brute force time (ms)	D&C time (ms)
10	45	10	4.5×	0.006	0.012
50	1,225	68	18.0×	0.14	0.078
100	4,950	135	36.7×	0.57	0.17
500	124,750	564	221.2×	15.4	1.11
1,000	499,500	1,128	442.8×	61.5	2.44
2,000	1,999,000	2,297	870.3×	250.3	5.42
4,000	7,998,000	4,625	1729.3×	992.3	11.6

Finer measurement near the crossover (average over 20 random datasets):

n	Brute force time (ms)	D&C time (ms)	Faster
10	0.006	0.011	Brute force
15	0.013	0.019	Brute force
20	0.023	0.025	Brute force
25	0.035	0.031	D&C
30	0.052	0.043	D&C
40	0.092	0.057	D&C

- **Growth:** brute force grows with n^2: doubling n multiplies the calculations by 4 (499,500 → 1,999,000 for n = 1,000 → 2,000). D&C grows a little faster than linearly: doubling n gives about 2.0× (1,128 → 2,297 → 4,625), because the strip stage compares each point with only a few neighbours.

- **Distance calculations** favour D&C from n = 10 (45 against 10) and at n = 4,000 the saving is 1,729 times.
- **Execution time:** D&C is slower for n ≤ 20 (recursion, sorting and merging overhead) and faster from about **n = 25** (n = 25: 0.031 against 0.035 ms; n = 100: 0.17 against 0.57 ms; n = 1,000: 2.4 against 61.5 ms, 25 times faster; n = 4,000: 11.6 ms against 992 ms, 85 times faster). So D&C becomes practically advantageous at roughly 25 points in this implementation.

Cross-Understanding Answers

Student B: why does the strip/boundary stage not require comparison with every other point? Only points within distance δ of the dividing line can be part of a straddling pair closer than δ. Within the strip, take a point p. A closer partner must lie in a rectangle of height δ above p and width 2δ. Each half of that rectangle (a δ × δ square) holds only points of the same half, and these points are at least δ apart from each other (δ is the smallest distance within that half), so at most 4 points fit in each δ × δ square; at most 8 points (7 other than p) can be in the whole rectangle. So each point is compared with at most 7 following points in y-order, not with all the others, and the strip work is O(n).

Student A: what experimental evidence demonstrates that the D&C strategy has actually reduced computational work? The exact counts of distance calculations: for n = 4,000, 4,625 (D&C) against 7,998,000 (brute force), a reduction of 1,729 times, and the growth between sizes (×4 for brute force, ×2 for D&C when n doubles). The execution times agree for n above about 25. The count is a machine-independent measure, which is why it is the primary evidence; the times show that the saved work is larger than the overhead of recursion.

Q6 – Advanced Divide-and-Conquer Mini-Investigations

Student A (Deepak S) – Median of Medians

Task: find the k-th smallest element (here the median, k = n/2) of an unsorted array without sorting it.
Counting convention: one comparison for every element at every partition pass, plus the exact comparisons of the insertion sort of each group of 5; the same convention is used for all methods below.

Algorithm and explanation

```
Select(A, k):
    if |A| ≤ 5: sort A and return the k-th
    split A into  n/5  groups of 5; take the median of each group (sort 5 elements)
    p ← Select(medians,  |medians|/2 )            // median of the medians: the pivot
    partition A into  L (< p),  E (= p),  G (> p)
    if k ≤ |L|:  return Select(L, k);   if k ≤ |L|+|E|:  return p;   else return
Select(G, k − |L| − |E|)
```

- **Grouping strategy:** groups of 5, because a group of 5 can be sorted with a few comparisons and gives a small median that is guaranteed to be above 2 and below 2 other elements of the group.
- **How the pivot is determined:** the median of each group is found; the pivot is the median of these $\lfloor n/5 \rfloor$ medians, found by a recursive call of the same algorithm.
- **Why consistently disastrous partitions are prevented:** about half of the group medians are ≤ p, and each of those groups has two more elements below its median, so at least 3 × (n/10) = 3n/10 elements are ≤ p (and similarly ≥ p). Therefore, whichever side is chosen, the recursion continues with at most **7n/10** elements. No input, however arranged, can make the pivot extreme.
- **Worst-case guarantee:** T(n) ≤ T(n/5) + T(7n/10) + cn. Because 1/5 + 7/10 = 9/10 < 1, the work per level shrinks geometrically and T(n) ≤ 10cn = **O(n)** in the worst case. (With groups of 3 the sum is 1/3 + 2/3 = 1, which gives O(n log n); that is why 5 is the smallest practical group size.)

Comparison with QuickSelect (n = median rank)

n	Input	Median of Medians	QuickSelect: random pivot	QuickSelect: first element pivot
1,000	random	7,205	1,870	3,015
	sorted	5,260	3,849	375,750
2,000	random	15,223	4,984	6,625
	sorted	10,909	6,234	1,501,500
4,000	random	30,757	23,682	13,269
	sorted	22,311	13,711	6,003,000
100,000	random	831,407	586,049	not run
	sorted	582,894	296,639	not run

- **Deterministic poor-pivot strategy:** with the first element as pivot, sorted input is the worst case: the pivot is always the minimum and each pass removes one element only: 375,750 → 1,501,500 → 6,003,000 comparisons for n = 1,000 → 2,000 → 4,000 (×4 each time, ≈ $3n^2/8 = \Theta(n^2)$). Median of Medians needed 5,260 → 10,909 → 22,311 for the same inputs (≈ 5.3n–5.6n, linear: ×2 each time), i.e. 269 times fewer at n = 4,000.
- **Randomisation reduces but does not eliminate poor partitions:** a random pivot lies in the outer 10% on each side with probability 20% (measured: 19.9% of 100,000 draws), so unlucky pivots do occur. Over 300 runs on sorted input with n = 4,000, random QuickSelect needed between 6,292 and 26,749 comparisons (median 12,799, mean 13,253, 95th percentile 20,229), a four-fold spread; the expected time is O(n), but there is no worst-case guarantee, only a very low probability of a long run.
- **Practical cost:** the guarantee is not free. Median of Medians used about 5n to 8n comparisons (n = 100,000: 582,894 on sorted and 831,407 on random input) while random QuickSelect used about 3.0n and 5.9n (296,639 and 586,049). Random QuickSelect is usually faster, and Median of Medians is chosen when a worst-case guarantee is needed (or as the fallback of an introspective algorithm).

Student B verification – why is pivot quality central to selection algorithms?

A selection algorithm keeps only one side of the partition. Its work is therefore the sum of the sizes of the sides it keeps: if each pivot discards a constant fraction of the elements, the sizes form a geometric series $n + \alpha n + \alpha^2 n + \ldots = O(n)$; if each pivot discards only O(1) elements, the sum is $n + (n-1) + (n-2) + \ldots = \Theta(n^2)$. The pivot is the only thing that decides which of the two happens, and our results show it: 6,003,000 against 22,311 comparisons for the same n = 4,000 sorted input.

Student B – Meet-in-the-Middle (Subset Sum)

Problem: given n positive integers and a target T, does some subset sum to T? Straightforward enumeration checks all 2^n subsets.

- **How the problem is divided:** the set is split into two halves S_1 and S_2 of n/2 numbers each. A subset of the whole set is a subset of S_1 together with a subset of S_2.
- **Generating subset results:** enumerate all 2^(n/2) subset sums of S_1 (list L) and of S_2 (list M).
- **Combining:** a subset with total T exists iff there is s L and t M with s + t = T. Sort M and, for every s in L, binary search for T − s (alternatively use two pointers on both sorted lists).
- **Search space:** 2^(n/2) + 2^(n/2) = 2^(n/2 + 1) generated sums (the 2^n combinations are covered implicitly), instead of 2^n subsets. Time: O(2^(n/2)·n) (the sorting and the searches).

- **Additional memory:** both lists, 2 × 2^(n/2) numbers, i.e. O(2^(n/2)) (for n = 40: 2,097,152 numbers ≈ 16 MB as 8-byte integers). Plain enumeration needs only O(n) memory if the subset sums are generated one at a time.

```
MeetInMiddle(w[1..n], T):
    L ← all subset sums of w[1..n/2];   M ← sorted(all subset sums of w[n/2+1..n])
    for s in L:  if (T − s) is in M (binary search):  return TRUE
    return FALSE
```

Both methods were checked against each other on 200 random solvable and unsolvable instances. Experiment (random weights between 10^5 and 10^6; unreachable target, which is the worst case because the whole search is performed):

n	Subsets 2^n	Meet-in-the-middle: sums generated	Reduction	Enumeration time (ms)	Meet-in-the-middle time (ms)
16	65,536	512	128×	3.96	0.14
20	1,048,576	2,048	512×	114.6	0.34
22	4,194,304	4,096	1,024×	575.9	0.33
24	16,777,216	8,192	2,048×	2,368	1.00
30	1,073,741,824	65,536	16,384×	not run	4.48
36	68,719,476,736	524,288	131,072×	not run	45.0
40	1,099,511,627,776	2,097,152	524,288×	not run	238.6

- **Compared with 2^n enumeration:** at n = 24 enumeration took 2.4 s against 1.0 ms for meet-in-the-middle (about 2,400 times faster). Enumeration was not run beyond n = 24. Its measured rate (about 141 ns per subset) would need about 150 s at n = 30 and roughly 43 hours at n = 40 (an estimate), but meet-in-the-middle solved n = 40 in 0.24 s.
- **Time–memory trade-off:** the method gains time by remembering sub-results: the time falls from 2^n to about 2^(n/2) at the cost of storing 2^(n/2) sums. Memory becomes the limit: n = 60 needs 2 × 2^30 ≈ 2 billion numbers (about 17 GB) even though the time (≈ 10^9 operations) would be acceptable. Memory can be reduced to O(2^(n/4)) with the Schroeppel–Shamir variant at the same time complexity.

Student A verification – why can Meet-in-the-Middle reduce computational work even though it introduces additional storage?

Because it replaces one very large search by two small ones plus a fast lookup: the stored lists are the square root of the original search space (2^(n/2) against 2^n), and storing a result is far cheaper than recomputing it repeatedly. The extra memory is exponentially smaller than the time saved: for n = 40 it holds about 2 million numbers (16 MB) and avoids about 10^{12} subset checks.

Q7 – Design Divide and Conquer from First Principles

Problem used (assumption). *The Day-4 problem statement was not included in the assignment sheet; the* ***Maximum Subarray Sum*** *problem was therefore chosen as the design problem: given an array a[1..n] of integers (positive and negative), find the largest sum of any non-empty contiguous block. Example: [−2, 1, −3, 4, −1, 2, 1, −5, 4] has maximum 6 (block 4, −1, 2, 1). If your Day-4 problem is a different one, the structure of this answer (Stages 1 to 6) can be reused.*

Stage 1 – Independent Design Proposals (retained as first written)

Student A (Deepak S) – initial proposal

- **Decomposition:** cut the array in the middle into a left half and a right half.
- **Base case:** one element: the answer is that element.
- **Information needed from a subproblem:** its maximum subarray sum.
- **Combine strategy:** the answer is the maximum of three values: left answer, right answer, and the best block that crosses the middle. To get the crossing block, start from the middle and scan left to find the best sum ending at the middle, then scan right to find the best sum starting after the middle, and add the two.

Student B – initial proposal

- **Decomposition:** cut the array into two halves, as above.
- **Base case:** an empty array returns 0 and a single element returns itself.
- **Information needed from a subproblem:** the best sum, and (after thought) the total sum of the segment.
- **Combine strategy:** answer = max(best of left, best of right, total of left + total of right), i.e. also consider joining the two halves completely.

Stage 2 – Comparison of the Two Designs

Aspect	Student A	Student B
Where they agree	Cut in the middle; two recursive calls; the answer is the best of "left only", "right only" or "crossing".	Same decomposition and the same basic idea of a three-way maximum.
Where they differ	Crossing block found by a linear scan outward from the middle.	Crossing block approximated by joining the two complete halves; no scan, but returns more information.
Strengths	Correct: handles every crossing block. Easy to prove.	Constant-time combine; the idea of returning extra data (total) from each subproblem is useful.
Weaknesses	Combine costs O(n) at each level, so T(n) = 2T(n/2) + n = Θ(n log n); the array is scanned again at every level.	Incorrect: a crossing block usually ends at some inner position of each half, not at the far end (for [−5, 4, 4, −5] it returns 4, but the answer is 8: the block 4, 4 crosses the middle without reaching the ends). The returned data is too little: the total alone is not enough.
Missing cases / information	Empty or all-negative halves (the crossing scan must include at least one element on each side).	Needs the best block that **ends** at the right end of the left half and the best block that **starts** at the left end of the right half.
Ideas retained	The three-way maximum, and correct handling of the crossing block.	Return extra information from each subproblem so that the crossing block can be found in O(1).

How the proposals influenced the final solution: Student A's analysis gave the correct structure, Student B's idea of returning more than the best sum provided the way to make the combine step constant-time. Combining them: each call returns the best suffix of its left part and the best prefix of its right part (the best blocks that touch the boundary), so the crossing block is computed from two stored numbers instead of a scan.

Stage 3 – Joint Final Design

- **Decomposition:** split a[lo..hi] at mid.
- **Subproblems:** the left half a[lo..mid] and the right half a[mid+1..hi], independent of each other.

- **Base case:** a single element x returns (total = x, best prefix = x, best suffix = x, best = x).
- **Information returned:** a 4-tuple (total, prefix, suffix, best) where total = sum of the segment, prefix = the best sum of a block starting at the left end, suffix = the best sum of a block ending at the right end, best = the maximum subarray sum of the segment.
- **Combine** (L = left result, R = right result): total = L.total + R.total; prefix = max(L.prefix, L.total + R.prefix); suffix = max(R.suffix, R.total + L.suffix); best = max(L.best, R.best, L.suffix + R.prefix). The crossing block is L.suffix + R.prefix.

```
MaxSub(a, lo, hi):
    if lo = hi:  return (a[lo], a[lo], a[lo], a[lo])            // (total, prefix,
suffix, best)
    mid ← (lo + hi) / 2
    L ← MaxSub(a, lo, mid);   R ← MaxSub(a, mid+1, hi)
    total  ← L.total + R.total
    prefix ← max(L.prefix, L.total + R.prefix)
    suffix ← max(R.suffix, R.total + L.suffix)
    best   ← max(L.best, R.best, L.suffix + R.prefix)
    return (total, prefix, suffix, best)
```

Trace on [−2, 1, −3, 4, −1, 2, 1, −5, 4] (split after index 4): the left part [−2, 1, −3, 4, −1] returns (total −1, prefix −1, suffix 3, best 4); the right part [2, 1, −5, 4] returns (total 2, prefix 3, suffix 4, best 4). Combine: best = max(4, 4, L.suffix + R.prefix = 3 + 3 = 6) = **6**, the block 4, −1, 2, 1, which is correct. The new total is −1 + 2 = 1.

Stage 4 – Validation

Student A (Deepak S) – Mathematical Validation

Recurrence: T(n) = 2T(n/2) + c, T(1) = 0. The term 2T(n/2) is the two recursive calls on half-size parts; c is the cost of the combine step, which is constant because it only uses the four stored numbers of each half: 4 comparisons (2 for best, 1 for prefix, 1 for suffix) and 4 additions, independent of n. The base case costs nothing.

Solution: counting comparisons (c = 4), T(n) = 2T(n/2) + 4 gives T(n) = 4(n − 1) for n = 2^k (check by substitution: 2·4(n/2 − 1) + 4 = 4n − 4). By the Master Theorem: a = 2, b = 2, f(n) = Θ(1) = O(n^(1−ε)), Case 1, so **T(n) = Θ(n)**. Compare: Student A's original combine with a scan gives T(n) = 2T(n/2) + n = Θ(n log n); brute force examines n(n + 1)/2 blocks, Θ(n²). The extra memory is O(log n) for the recursion stack.

Student B – Experimental Validation

Correctness test cases (all passed; results compared with brute force and with Kadane's linear algorithm):

Test case	Array	Result
single element	[5]	5
single negative	[-3]	-3
all negative	[-4, -1, -7, -2]	-1
all positive	[3, 1, 4, 1, 5]	14
classic mixed	[-2, 1, -3, 4, -1, 2, 1, -5, 4]	6
crossing midpoint	[5, -1, -1, 5]	8
zeros and negatives	[0, -1, 0, -2, 0]	0

In addition, 2,000 random arrays (length 1–40, values −20 … 20) gave identical answers for the D&C, brute-force and Kadane methods. The cases cover the base case, an all-negative array (the answer must be the largest single element, not 0), an all-positive array, the usual mixed array and a block crossing the midpoint.

Instrumented comparison (random arrays of values −50 … 50; "comparisons" counted in the combine step; brute force examined all n(n + 1)/2 blocks):

n	D&C comparisons	Predicted 4(n−1)	Brute-force block sums	D&C time (ms)	Brute-force time (ms)
2	4	4	3	0.001	0.001
4	12	12	10	0.001	0.001
8	28	28	36	0.003	0.002
16	60	60	136	0.006	0.005
64	252	252	2,080	0.026	0.075
256	1,020	1,020	32,896	0.11	1.25
1,024	4,092	4,092	524,800	0.47	23.3
4,096	16,380	16,380	8,390,656	1.90	376.3
16,384	65,532	65,532	134,225,920	7.65	6,090

Evidence. The observed count equals 4(n − 1) exactly for every n, which confirms the recurrence. Quadrupling n (4,096 → 16,384) multiplies the D&C time by 4 (1.9 → 7.6 ms) but the brute-force time by 16 (376 → 6,090 ms). Brute force is faster for n ≤ 16 (0.005 against 0.006 ms) and D&C is clearly faster from n = 64 (0.026 against 0.075 ms); at n = 16,384, D&C is about 800 times faster.

Stage 5 – Cross-Defence

Student A: what experimental evidence supports the group's design? (1) Correctness: on all seven test cases (including the crossing and all-negative cases) and on 2,000 random arrays, the answer equals the brute-force and Kadane results. (2) Efficiency: the measured comparison counts are exactly 4(n − 1), growing linearly with n, whereas the brute-force baseline grows quadratically (n = 16,384: 65,532 comparisons against 134,225,920 block sums); the running time confirms it (7.6 ms against 6,090 ms).

Student B: how does the recurrence arise from the group's decomposition? The decomposition cuts the array into two halves of size n/2, each handled by one recursive call; this gives the term 2T(n/2). The combine step uses only the four stored numbers of each half and a fixed number of additions and comparisons, independent of n, so it contributes a constant c. The base case (one element) costs nothing. Together: T(n) = 2T(n/2) + c, T(1) = 0. The recursion tree has n leaves and n − 1 internal nodes, each costing c, hence about c(n − 1) = Θ(n).

Stage 6 – Joint Evaluation

- **Why the problem suits D&C:** the problem splits into independent subproblems of the same kind (a half-array has the same problem), the answer for the whole is either entirely inside a half or crosses the middle, and the crossing case can be computed from a small summary of each half. The subproblems do not overlap, so nothing is computed twice.
- **Computational work reduced by decomposition:** the brute-force method re-adds overlapping blocks ($\Theta(n^2)$ block sums). The summaries (total, prefix, suffix, best) make it unnecessary to look at the elements of a half again, reducing the work to Θ(n).
- **Additional costs of recursion:** function-call overhead for 2n − 1 calls, an O(log n) stack, and tuple creation per call; at small sizes (n ≤ 16) the brute-force method was faster in our timings. Practical programs cut off the recursion at small sizes.
- **When D&C might not be appropriate:** when a simpler algorithm achieves the same complexity (here, Kadane's algorithm is Θ(n) with O(1) memory and no recursion, so for a single sequential pass D&C offers no asymptotic gain; it is still useful when the work should be parallelised, for data stored in a

segment tree, or for answering many range queries); when subproblems overlap (dynamic programming is then better); when the combine step is as costly as the original problem; when the problem cannot be split into independent parts of balanced size; or when the input is so small that overhead dominates.

Conclusion

What characteristics should an algorithm designer look for before choosing Divide and Conquer for a new problem? (1) The problem can be split into **smaller instances of the same problem**, preferably of balanced size. (2) The subproblems are **independent** (no overlap). (3) There is a **cheap combine step**: the solution for the whole can be assembled from the sub-solutions, possibly after deciding what extra information each sub-solution should return. (4) A **simple base case** exists. (5) The recurrence is favourable: the total work of the divide and combine steps per level must be smaller than the work saved (e.g. $T(n) = 2T(n/2) + O(n)$ or better), the Master Theorem is a quick test. (6) Overheads (recursion, extra memory, cut-off size) are acceptable, and no simpler algorithm is as good.

Contribution Record

Question	Student A (Deepak S) – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1	Max/Min design and comparison analysis; binary search experiments	Max/Min experimental validation; binary search elimination logic	Strategy, base cases and implementation of both algorithms
Q2	Quick Sort investigation: balanced/unbalanced inputs, partition sizes, depth	Comparative analysis of Merge Sort and Quick Sort decomposition	Both implementations, pivot choice, experiments, joint design question
Q3	Karatsuba derivation, recurrence and complexity	Crossover experiments and overhead analysis	Standard and Karatsuba implementations, joint conclusion
Q4	Experiments, counts, timing, overhead and cache analysis	Algebraic design, correctness proof, padding	Both implementations, correctness verification, engineering decision
Q5	Decomposition design of the D&C algorithm	Datasets, measurements and comparative evidence	Brute-force review, implementation and cross-understanding answers
Q6	Median of Medians (primary); verification of Meet-in-the-Middle	Meet-in-the-Middle (primary); verification of pivot quality	Review of each other's sections
Q7	Independent proposal (Stage 1); mathematical validation	Independent proposal (Stage 1); experimental validation	Stages 2, 3, 5 and 6

Primary contribution does not imply exclusive responsibility; both students remain accountable for the complete assignment.

Student A signature: _____ Student B signature: _____ Date: _____

Appendix – Source Code (experiments.py)

Complete Python program used for all experiments above (Python 3, NumPy). It writes the measurements to a results file from which the tables in this document were produced.

```
import random, time, json, math, sys, itertools
from collections import deque
import numpy as np

random.seed(42)
R = {}

def timeit(f, reps=3):
    best = 1e9
    for _ in range(reps):
        t = time.perf_counter(); f(); best = min(best, time.perf_counter() - t)
    return best * 1000  # ms

# ---------------------------------------------------------------- Q1 Max/Min
def maxmin(a, lo, hi, c):
    if lo == hi:
        return a[lo], a[lo]
    if hi == lo + 1:
        c[0] += 1
        return (a[lo], a[hi]) if a[lo] < a[hi] else (a[hi], a[lo])
    mid = (lo + hi) // 2
    mn1, mx1 = maxmin(a, lo, mid, c)
    mn2, mx2 = maxmin(a, mid + 1, hi, c)
    c[0] += 2
    return (mn1 if mn1 < mn2 else mn2), (mx1 if mx1 > mx2 else mx2)

def maxmin_naive(a, c):
    mn = mx = a[0]
    for x in a[1:]:
        c[0] += 1
        if x < mn: mn = x
        c[0] += 1
        if x > mx: mx = x
    return mn, mx

q1 = []
for n in [2, 4, 8, 16, 64, 256, 1024, 4096, 65536, 100, 1000, 10000, 100000]:
    data = {
        'random': [random.randint(0, 10**6) for _ in range(n)],
        'sorted': list(range(n)),
        'reverse': list(range(n, 0, -1)),
        'all equal': [7] * n,
    }
    counts = {}
    for name, a in data.items():
        c = [0]; mn, mx = maxmin(a, 0, n - 1, c)
        assert mn == min(a) and mx == max(a)
        counts[name] = c[0]
    c2 = [0]; maxmin_naive(data['random'], c2)
    q1.append(dict(n=n, dc=counts, pred=math.ceil(3 * n / 2) - 2, naive=c2[0]))
R['q1'] = q1

# ---------------------------------------------------------------- Q1 Binary search
def bsearch(a, key):
    lo, hi, p = 0, len(a) - 1, 0
    while lo <= hi:
        mid = (lo + hi) // 2
        p += 1
        if a[mid] == key: return mid, p
        if a[mid] < key: lo = mid + 1
        else: hi = mid - 1
    return -1, p

q1b = []
for n in [7, 15, 100, 1000, 10**4, 10**5, 10**6]:
    a = [2 * i for i in range(n)]          # even numbers -> odd keys are absent
    keys = range(n) if n <= 10**5 else random.sample(range(n), 20000)
    ps = [bsearch(a, a[k])[1] for k in keys]
    us = [bsearch(a, 2 * k + 1)[1] for k in keys]
    q1b.append(dict(n=n, succ_avg=sum(ps) / len(ps), succ_max=max(ps), uns_avg=sum(us) / len(us),
                    uns_max=max(us), bound=math.floor(math.log2(n)) + 1))
```

```
R['q1b'] = q1b

# ---------------------------------------------------------------- Q2 Sorting
def qsort_stats(arr, strategy='first'):
    q = deque([(arr, 1)]); comps = 0; maxd = 0; log = []
    while q:
        a, d = q.popleft()
        maxd = max(maxd, d)
        if len(a) <= 1: continue
        if strategy == 'first':
            i = 0
        else:  # median of three
            m = len(a) // 2
            trio = sorted([(a[0], 0), (a[m], m), (a[-1], len(a) - 1)])
            comps += 3; i = trio[1][1]
        p = a[i]; rest = a[:i] + a[i + 1:]
        comps += len(rest)
        less = [x for x in rest if x < p]; gr = [x for x in rest if x > p]
        if len(log) < 6: log.append((len(a), len(less), len(gr)))
        q.append((less, d + 1)); q.append((gr, d + 1))
    return comps, maxd, log

def merge_stats(a):
    comps = [0]; maxd = [0]
    def ms(x, d):
        maxd[0] = max(maxd[0], d)
        if len(x) <= 1: return x
        m = len(x) // 2
        l = ms(x[:m], d + 1); r = ms(x[m:], d + 1)
        out = []; i = j = 0
        while i < len(l) and j < len(r):
            comps[0] += 1
            if l[i] <= r[j]: out.append(l[i]); i += 1
            else: out.append(r[j]); j += 1
        out += l[i:] + r[j:]
        return out
    res = ms(a, 1)
    assert res == sorted(a)
    return comps[0], maxd[0]

def balanced_perm(lo, hi):   # permutation of lo..hi on which first-pivot QS always splits in the middle
    if lo > hi: return []
    m = (lo + hi) // 2
    return [m] + balanced_perm(lo, m - 1) + balanced_perm(m + 1, hi)

q2 = []
n = 1023
inputs = {'balanced (constructed)': balanced_perm(1, n), 'random': random.sample(range(1, n + 1), n),
          'sorted ascending': list(range(1, n + 1)), 'sorted descending': list(range(n, 0, -1))}
for name, a in inputs.items():
    c, d, log = qsort_stats(a, 'first')
    c3, d3, _ = qsort_stats(a, 'med3')
    mc, md = merge_stats(a)
    q2.append(dict(name=name, qs_comps=c, qs_depth=d, qs_log=log, m3_comps=c3, m3_depth=d3, ms_comps=mc, ms_depth=md))
R['q2'] = q2; R['q2_n'] = n
q2s = []
for n in [250, 500, 1000, 2000, 4000]:
    a = list(range(n)); c, d, _ = qsort_stats(a, 'first'); mc, md = merge_stats(a)
    r = random.sample(range(n), n); rc, rd, _ = qsort_stats(r, 'first')
    q2s.append(dict(n=n, sorted_comps=c, sorted_depth=d, rand_comps=rc, rand_depth=rd, ms=mc, msd=md, quad=n * (n - 1)
// 2,
                    nlogn=round(n * math.log2(n))))
R['q2s'] = q2s

# ---------------------------------------------------------------- Q3 Karatsuba (decimal digit lists, little-endian)
MUL = [0]
def norm(x):
    x = x[:]; c = 0
    for i in range(len(x)):
        v = x[i] + c; x[i] = v % 10; c = v // 10
    while c: x.append(c % 10); c //= 10
    while len(x) > 1 and x[-1] == 0: x.pop()
    return x
def add_raw(a, b):          # digit-wise sum, no carry propagation (digits may reach 18, length does not grow)
    n = max(len(a), len(b)); r = [0] * n
    for i in range(n): r[i] = (a[i] if i < len(a) else 0) + (b[i] if i < len(b) else 0)
    return r
def sub(a, b):              # a >= b, both normalised
    r = a[:]; br = 0
    for i in range(len(r)):
```

```
        v = r[i] - (b[i] if i < len(b) else 0) - br
        if v < 0: v += 10; br = 1
        else: br = 0
        r[i] = v
    while len(r) > 1 and r[-1] == 0: r.pop()
    return r
def school(a, b):
    r = [0] * (len(a) + len(b) + 14)
    for i, x in enumerate(a):
        c = 0
        for j, y in enumerate(b):
            v = r[i + j] + x * y + c; r[i + j] = v % 10; c = v // 10
        k = i + len(b)
        while c: v = r[k] + c; r[k] = v % 10; c = v // 10; k += 1
    MUL[0] += len(a) * len(b)
    return norm(r)
def kara(a, b, th):
    n = max(len(a), len(b))
    if n <= th: return school(a, b)
    a = a + [0] * (n - len(a)); b = b + [0] * (n - len(b))
    m = n // 2
    a0, a1, b0, b1 = a[:m], a[m:], b[:m], b[m:]
    z0 = kara(a0, b0, th); z2 = kara(a1, b1, th)
    z1 = sub(sub(kara(add_raw(a0, a1), add_raw(b0, b1), th), z0), z2)
    res = [0] * (len(z2) + 2 * m + 2)
    for i, v in enumerate(z0): res[i] += v
    for i, v in enumerate(z1): res[i + m] += v
    for i, v in enumerate(z2): res[i + 2 * m] += v
    return norm(res)
def tonum(x): return int(''.join(map(str, reversed(x)))) if x else 0
def todigits(n): return [int(c) for c in reversed(str(n))]


q3 = []
for n in [4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048]:
    A = [random.randint(0, 9) for _ in range(n - 1)] + [random.randint(1, 9)]
    B = [random.randint(0, 9) for _ in range(n - 1)] + [random.randint(1, 9)]
    ref = tonum(A) * tonum(B)
    row = dict(n=n)
    MUL[0] = 0; assert tonum(school(A, B)) == ref; row['school_mul'] = MUL[0]
    MUL[0] = 0; assert tonum(kara(A, B, 1)) == ref; row['kara_mul'] = MUL[0]
    reps = 3 if n <= 512 else 2
    row['t_school'] = timeit(lambda: school(A, B), reps)
    row['t_kara1'] = timeit(lambda: kara(A, B, 1), reps)
    row['t_kara16'] = timeit(lambda: kara(A, B, 16), reps)
    row['t_kara64'] = timeit(lambda: kara(A, B, 64), reps)
    q3.append(row)
R['q3'] = q3


# ------------------------------------------------------------------ Q4 Strassen (pure python lists)
SM = [0]
def madd(A, B): return [[x + y for x, y in zip(r, s)] for r, s in zip(A, B)]
def msub(A, B): return [[x - y for x, y in zip(r, s)] for r, s in zip(A, B)]
def conv(A, B):
    n = len(A); SM[0] += n ** 3
    Bt = list(zip(*B))
    return [[sum(x * y for x, y in zip(r, c)) for c in Bt] for r in A]
def blocks(M):
    h = len(M) // 2
    return [r[:h] for r in M[:h]], [r[h:] for r in M[:h]], [r[:h] for r in M[h:]], [r[h:] for r in M[h:]]
def join(C11, C12, C21, C22):
    return [a + b for a, b in zip(C11, C12)] + [a + b for a, b in zip(C21, C22)]
def block8(A, B, cutoff):
    n = len(A)
    if n <= cutoff: return conv(A, B)
    a11, a12, a21, a22 = blocks(A); b11, b12, b21, b22 = blocks(B)
    return join(madd(block8(a11, b11, cutoff), block8(a12, b21, cutoff)), madd(block8(a11, b12, cutoff), block8(a12,
b22, cutoff)),
                madd(block8(a21, b11, cutoff), block8(a22, b21, cutoff)), madd(block8(a21, b12, cutoff), block8(a22,
b22, cutoff)))
def strassen(A, B, cutoff):
    n = len(A)
    if n <= cutoff: return conv(A, B)
    a11, a12, a21, a22 = blocks(A); b11, b12, b21, b22 = blocks(B)
    M1 = strassen(madd(a11, a22), madd(b11, b22), cutoff); M2 = strassen(madd(a21, a22), b11, cutoff)
    M3 = strassen(a11, msub(b12, b22), cutoff); M4 = strassen(a22, msub(b21, b11), cutoff)
    M5 = strassen(madd(a11, a12), b22, cutoff); M6 = strassen(msub(a21, a11), madd(b11, b12), cutoff)
    M7 = strassen(msub(a12, a22), madd(b21, b22), cutoff)
    return join(msub(madd(M1, M4), msub(M5, M7)), madd(M3, M5), madd(M2, M4), madd(msub(M1, M2), madd(M3, M6)))
def pad(M, m):
    n = len(M); return [r + [0] * (m - n) for r in M] + [[0] * m for _ in range(m - n)]
def strassen_any(A, B, cutoff):
    n = len(A); m = 1
```

```
    while m < n: m *= 2
    C = strassen(pad(A, m), pad(B, m), cutoff)
    return [r[:n] for r in C[:n]]

# correctness
chk = {}
A0, B0 = [[1, 7], [3, 5]], [[1, 3], [7, 5]]
chk['sample'] = strassen(A0, B0, 1)
for n in [1, 2, 3, 5, 7, 10, 33, 100]:
    A = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    ref = (np.array(A) @ np.array(B)).tolist()
    assert strassen_any(A, B, 1) == ref and strassen_any(A, B, 4) == ref
    if n & (n - 1) == 0 or n < 4: assert block8(pad(A, 1 if n == 1 else 2 ** math.ceil(math.log2(n))), pad(B, 1 if n
== 1 else 2 ** math.ceil(math.log2(n))), 1) is not None
chk['tested_sizes'] = [1, 2, 3, 5, 7, 10, 33, 100]
R['q4_check'] = chk

q4 = []
for n in [4, 8, 16, 32, 64, 128]:
    A = [[random.randint(0, 9) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(0, 9) for _ in range(n)] for _ in range(n)]
    row = dict(n=n, conv_mul=n ** 3, str_mul=7 ** int(math.log2(n)))
    SM[0] = 0; strassen(A, B, 8); row['hyb8_mul'] = SM[0]
    SM[0] = 0; strassen(A, B, 16); row['hyb16_mul'] = SM[0]
    reps = 3 if n <= 64 else 1
    row['t_conv'] = timeit(lambda: conv(A, B), reps)
    row['t_str1'] = timeit(lambda: strassen(A, B, 1), reps)
    row['t_str8'] = timeit(lambda: strassen(A, B, 8), reps)
    row['t_str16'] = timeit(lambda: strassen(A, B, 16), reps)
    q4.append(row)
R['q4'] = q4

# numpy / BLAS level
def np_str(A, B, cutoff):
    n = A.shape[0]
    if n <= cutoff: return A @ B
    h = n // 2
    a11, a12, a21, a22 = A[:h, :h], A[:h, h:], A[h:, :h], A[h:, h:]
    b11, b12, b21, b22 = B[:h, :h], B[:h, h:], B[h:, :h], B[h:, h:]
    M1 = np_str(a11 + a22, b11 + b22, cutoff); M2 = np_str(a21 + a22, b11, cutoff)
    M3 = np_str(a11, b12 - b22, cutoff); M4 = np_str(a22, b21 - b11, cutoff)
    M5 = np_str(a11 + a12, b22, cutoff); M6 = np_str(a21 - a11, b11 + b12, cutoff)
    M7 = np_str(a12 - a22, b21 + b22, cutoff)
    C = np.empty_like(A)
    C[:h, :h] = M1 + M4 - M5 + M7; C[:h, h:] = M3 + M5; C[h:, :h] = M2 + M4; C[h:, h:] = M1 - M2 + M3 + M6
    return C
q4n = []
rng = np.random.default_rng(1)
for n in [256, 512, 1024, 2048]:
    A = rng.random((n, n)); B = rng.random((n, n))
    row = dict(n=n)
    row['t_blas'] = timeit(lambda: A @ B, 2)
    for cut in [32, 64, 128, 256]:
        if cut < n:
            row['t_s%d' % cut] = timeit(lambda: np_str(A, B, cut), 2)
    row['err'] = float(np.abs(np_str(A, B, 128) - A @ B).max())
    q4n.append(row)
R['q4n'] = q4n

# ---------------------------------------------------------------- Q5 Closest pair
def brute(P):
    best = float('inf'); c = 0
    for i in range(len(P)):
        for j in range(i + 1, len(P)):
            c += 1
            d = (P[i][0] - P[j][0]) ** 2 + (P[i][1] - P[j][1]) ** 2
            if d < best: best = d
    return math.sqrt(best), c

def closest_dc(P):
    cnt = [0]
    Px = sorted(P)
    def d2(p, q):
        cnt[0] += 1
        return (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2
    def rec(lo, hi):          # returns (best squared distance, points sorted by y)
        n = hi - lo
        if n <= 3:
            pts = Px[lo:hi]; best = float('inf')
```

```
            for i in range(n):
                for j in range(i + 1, n): best = min(best, d2(pts[i], pts[j]))
            return best, sorted(pts, key=lambda p: p[1])
        mid = lo + n // 2; midx = Px[mid][0]
        dl, yl = rec(lo, mid); dr, yr = rec(mid, hi)
        d = min(dl, dr)
        ys = []; i = j = 0
        while i < len(yl) and j < len(yr):
            if yl[i][1] <= yr[j][1]: ys.append(yl[i]); i += 1
            else: ys.append(yr[j]); j += 1
        ys += yl[i:] + yr[j:]
        strip = [p for p in ys if (p[0] - midx) ** 2 < d]
        for i in range(len(strip)):
            for j in range(i + 1, len(strip)):
                if (strip[j][1] - strip[i][1]) ** 2 >= d: break
                d = min(d, d2(strip[i], strip[j]))
        return d, ys
    d, _ = rec(0, len(Px))
    return math.sqrt(d), cnt[0]


q5 = []
for n in [10, 50, 100, 500, 1000, 2000, 4000]:
    P = [(random.random() * 10**4, random.random() * 10**4) for _ in range(n)]
    b, cb = brute(P); d, cd = closest_dc(P)
    assert abs(b - d) < 1e-9
    reps = 3 if n <= 1000 else 1
    q5.append(dict(n=n, bc=cb, dc=cd, tb=timeit(lambda: brute(P), reps), td=timeit(lambda: closest_dc(P), reps),
                   ratio=cb / cd, dist=b))
R['q5'] = q5
# edge dataset: all points on a vertical line (strip stress)
P = [(5.0, float(random.randint(0, 10**6))) for _ in range(2000)]
b, cb = brute(P); d, cd = closest_dc(P); assert abs(b - d) < 1e-9
R['q5_line'] = dict(n=2000, bc=cb, dc=cd)

# ---------------------------------------------------------------- Q6 Selection
def insertion_sort_counted(g, c):
    g = g[:]
    for i in range(1, len(g)):
        k = g[i]; j = i - 1
        while j >= 0:
            c[0] += 1
            if g[j] > k: g[j + 1] = g[j]; j -= 1
            else: break
        g[j + 1] = k
    return g


def mom_select(a, k, c):    # k is 0-based rank
    if len(a) <= 5:
        return insertion_sort_counted(a, c)[k]
    meds = [insertion_sort_counted(a[i:i + 5], c)[len(a[i:i + 5]) // 2] for i in range(0, len(a), 5)]
    p = mom_select(meds, len(meds) // 2, c)
    c[0] += len(a)
    lo = [x for x in a if x < p]; eq = [x for x in a if x == p]; hi = [x for x in a if x > p]
    if k < len(lo): return mom_select(lo, k, c)
    if k < len(lo) + len(eq): return p
    return mom_select(hi, k - len(lo) - len(eq), c)


def quickselect(a, k, c, mode):
    while True:
        if len(a) == 1: return a[0]
        p = a[0] if mode == 'first' else random.choice(a)
        c[0] += len(a)
        lo = [x for x in a if x < p]; eq = [x for x in a if x == p]; hi = [x for x in a if x > p]
        if k < len(lo): a = lo
        elif k < len(lo) + len(eq): return p
        else: k -= len(lo) + len(eq); a = hi


q6 = []
for n in [1000, 2000, 4000, 100000]:
    rnd = random.sample(range(10 * n), n); srt = sorted(rnd); k = n // 2
    row = dict(n=n)
    for nm, a in [('random', rnd), ('sorted', srt)]:
        c = [0]; v = mom_select(a, k, c); assert v == srt[k]; row['mom_' + nm] = c[0]
        c = [0]; v = quickselect(a, k, c, 'rand'); assert v == srt[k]; row['qsr_' + nm] = c[0]
        if n <= 4000:
            c = [0]; v = quickselect(a, k, c, 'first'); assert v == srt[k]; row['qsf_' + nm] = c[0]
    q6.append(row)
R['q6'] = q6
# distribution of randomised quickselect (n = 4000, sorted input, 300 trials) and fraction of poor pivots
n = 4000; a = list(range(n)); cs = []
```

```
for _ in range(300):
    c = [0]; quickselect(a, n // 2, c, 'rand'); cs.append(c[0])
cs.sort()
R['q6_rand'] = dict(n=n, min=cs[0], median=cs[150], p95=cs[285], max=cs[-1], mean=sum(cs) / len(cs))
bad = sum(1 for _ in range(100000) if (lambda r: r < 0.1 * n or r > 0.9 * n)(random.randrange(n)))
R['q6_bad_frac'] = bad / 100000


# Meet in the middle subset sum
def brute_ss(w, T):
    sums = [0]; ops = 0
    for x in w:
        sums = sums + [s + x for s in sums]; ops += len(sums) // 2
    return T in set(sums), len(sums)


def mitm_ss(w, T):
    h = len(w) // 2
    def allsums(x):
        s = [0]
        for v in x: s = s + [t + v for t in s]
        return s
    L = allsums(w[:h]); Rr = sorted(allsums(w[h:]))
    import bisect
    ops = len(L) + len(Rr)
    for s in L:
        i = bisect.bisect_left(Rr, T - s); ops += 1
        if i < len(Rr) and Rr[i] == T - s: return True, len(L) + len(Rr), ops
    return False, len(L) + len(Rr), ops


q6m = []
for n in [16, 20, 22, 24, 30, 36, 40]:
    w = [random.randint(10**5, 10**6) for _ in range(n)]
    T = sum(w) // 2 + 1                                  # target that is (almost surely) unreachable -> worst case
    row = dict(n=n, subsets=2 ** n, mitm_entries=2 * 2 ** (n // 2))
    if n <= 24:
        t0 = time.perf_counter(); r1, _ = brute_ss(w, T); row['t_brute'] = (time.perf_counter() - t0) * 1000
    t0 = time.perf_counter(); r2, ent, ops = mitm_ss(w, T); row['t_mitm'] = (time.perf_counter() - t0) * 1000
    row['mitm_ops'] = ops
    if n <= 24: assert r1 == r2
    q6m.append(row)
# correctness with solvable instances
for _ in range(200):
    n = random.randint(4, 16); w = [random.randint(1, 50) for _ in range(n)]
    T = random.randint(0, sum(w))
    assert brute_ss(w, T)[0] == mitm_ss(w, T)[0]
R['q6m'] = q6m


# ------------------------------------------------------------------ Q7 Maximum subarray
def msub_dc(a, lo, hi, c):     # returns (total, best prefix, best suffix, best)
    if lo == hi: return a[lo], a[lo], a[lo], a[lo]
    mid = (lo + hi) // 2
    t1, p1, s1, b1 = msub_dc(a, lo, mid, c); t2, p2, s2, b2 = msub_dc(a, mid + 1, hi, c)
    c[0] += 4
    return t1 + t2, max(p1, t1 + p2), max(s2, t2 + s1), max(b1, b2, s1 + p2)
def msub_brute(a):
    best = a[0]; c = 0
    for i in range(len(a)):
        s = 0
        for j in range(i, len(a)):
            s += a[j]; c += 1
            if s > best: best = s
    return best, c
def kadane(a):
    best = cur = a[0]
    for x in a[1:]: cur = max(x, cur + x); best = max(best, cur)
    return best
tests = {'single element': [5], 'single negative': [-3], 'all negative': [-4, -1, -7, -2], 'all positive': [3, 1, 4,
1, 5],
         'classic mixed': [-2, 1, -3, 4, -1, 2, 1, -5, 4], 'crossing midpoint': [5, -1, -1, 5], 'zeros and negatives':
[0, -1, 0, -2, 0]}
q7t = []
for nm, a in tests.items():
    c = [0]; v = msub_dc(a, 0, len(a) - 1, c)[3]; bv = msub_brute(a)[0]
    assert v == bv == kadane(a); q7t.append((nm, a, v))
for _ in range(2000):
    n = random.randint(1, 40); a = [random.randint(-20, 20) for _ in range(n)]
    c = [0]; assert msub_dc(a, 0, n - 1, c)[3] == msub_brute(a)[0] == kadane(a)
R['q7t'] = q7t
q7 = []
for n in [2, 4, 8, 16, 64, 256, 1024, 4096, 16384]:
    a = [random.randint(-50, 50) for _ in range(n)]
    c = [0]; msub_dc(a, 0, n - 1, c)
```

```
    row = dict(n=n, dc=c[0], pred=4 * (n - 1), brute=n * (n + 1) // 2)
    reps = 3 if n <= 4096 else 1
    row['t_dc'] = timeit(lambda: msub_dc(a, 0, n - 1, [0]), reps)
    row['t_brute'] = timeit(lambda: msub_brute(a), reps) if n <= 16384 else None
    q7.append(row)
R['q7'] = q7

json.dump(R, open('/home/claude/results.json', 'w'), indent=1)
print('done')
```

Additional check – conventional block recursion (verify_block8.py)

Output: block8 verified for n = 2..32; multiplications = n^3.

```
# Extra check: conventional block recursion (8 block products) equals the NumPy product
import random, math
import numpy as np
SM = [0]
def madd(A, B): return [[x + y for x, y in zip(r, s)] for r, s in zip(A, B)]
def conv(A, B):
    n = len(A); SM[0] += n ** 3
    Bt = list(zip(*B))
    return [[sum(x * y for x, y in zip(r, c)) for c in Bt] for r in A]
def blocks(M):
    h = len(M) // 2
    return [r[:h] for r in M[:h]], [r[h:] for r in M[:h]], [r[:h] for r in M[h:]], [r[h:] for r in M[h:]]
def join(C11, C12, C21, C22):
    return [a + b for a, b in zip(C11, C12)] + [a + b for a, b in zip(C21, C22)]
def block8(A, B, cutoff):
    n = len(A)
    if n <= cutoff: return conv(A, B)
    a11, a12, a21, a22 = blocks(A); b11, b12, b21, b22 = blocks(B)
    return join(madd(block8(a11, b11, cutoff), block8(a12, b21, cutoff)), madd(block8(a11, b12, cutoff), block8(a12,
b22, cutoff)),
                madd(block8(a21, b11, cutoff), block8(a22, b21, cutoff)), madd(block8(a21, b12, cutoff), block8(a22,
b22, cutoff)))
for n in [2, 4, 8, 16, 32]:
    A = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    B = [[random.randint(-9, 9) for _ in range(n)] for _ in range(n)]
    SM[0] = 0
    assert block8(A, B, 1) == (np.array(A) @ np.array(B)).tolist()
    assert SM[0] == n ** 3        # eight sub-products per level -> n^3 scalar multiplications
print('block8 verified for n = 2..32; multiplications = n^3')
```